PROTOKOLL E-BEFISCHUNG · DEVELOPER HANDOVER

How four features actually work

Saving drafts, roles and review, reading the old PDF in, and printing a protocol out. Each feature is explained three times, each time deeper: a short story first, then the flow as a diagram, then the backend and database decisions behind it.

0 · The shared basis | 1 · Entwürfe speichern | 2 · Rollen und Review | 3 · PDF-Import | 4 · PDF-Export | Cheat sheet

READ THIS FIRST

0 · The shared basis

`features 2, 3a, 11b`

1 IN ONE MINUTE

Think of every protocol as **one folder**. On the front of the folder are a few labels the office needs to sort and find it: who it belongs to, what state it is in, the date, where the survey happened. Inside the folder is one big sheet with all of the roughly 338 answers.

In the database that folder is **one row in the** `submissions` **table**. The labels are normal columns. The big sheet is a single JSON column called `antworten`. All

four features below read or write that one row.

A few German words keep coming up. The code uses them on purpose, because they match the field names in the official FFS form and in FiaKa, the state database the data ends up in.

`Protokoll`
One electrofishing survey report.

`Entwurf`
A draft: a protocol that is still being filled in.

`antworten`
"Answers": the JSON document with every field.

`Absenden`
"Send off": submitting a protocol to FFS.

`Prüfung`
Review by FFS staff.

`Verlauf`
The history of status changes.

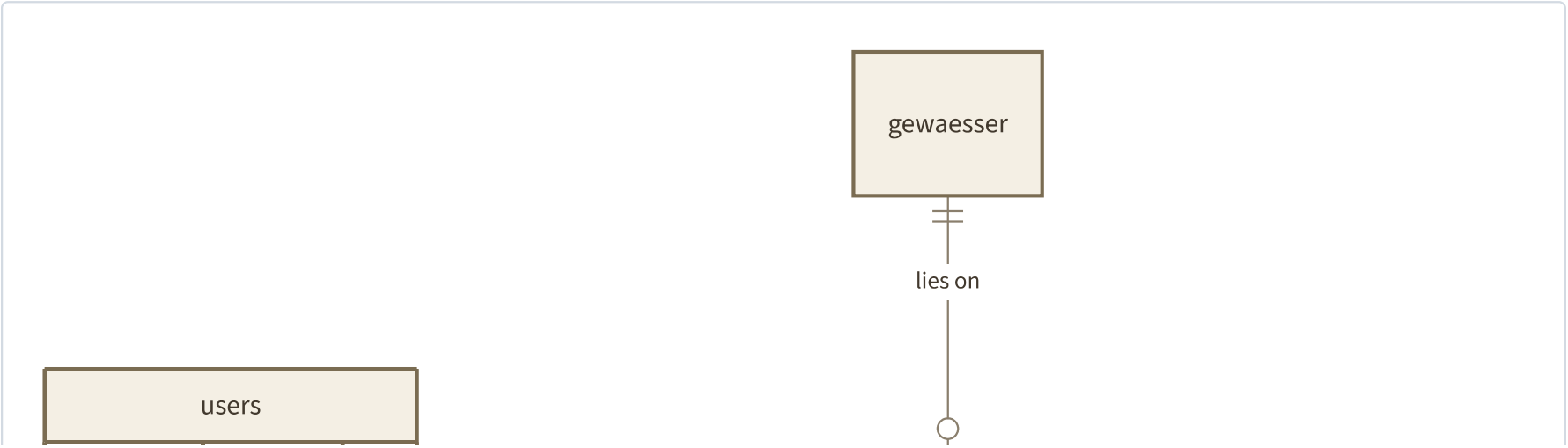
`Gewässer`
A water body (river, stream, lake).

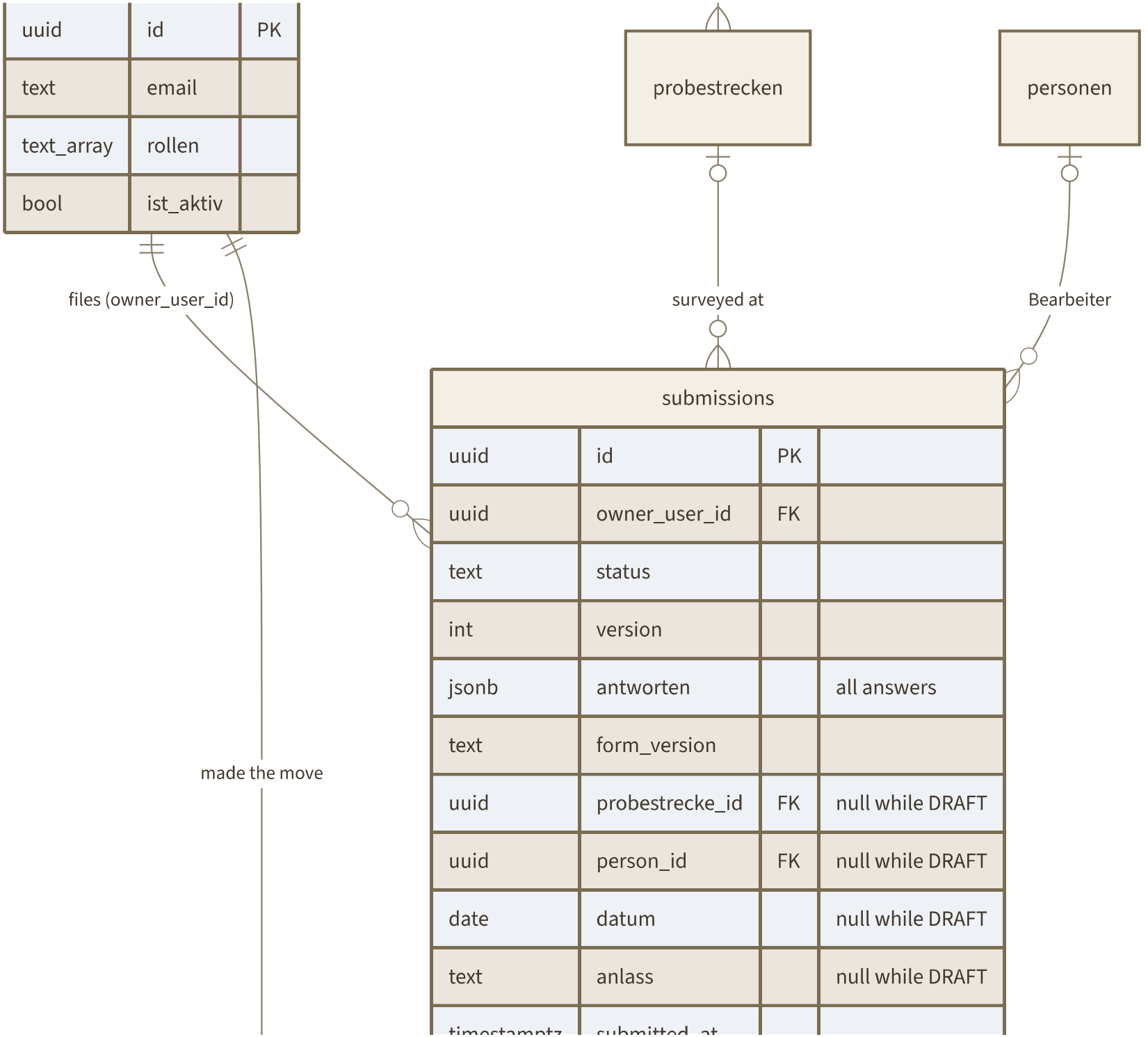
`Probestrecke`
The stretch of water that was fished.

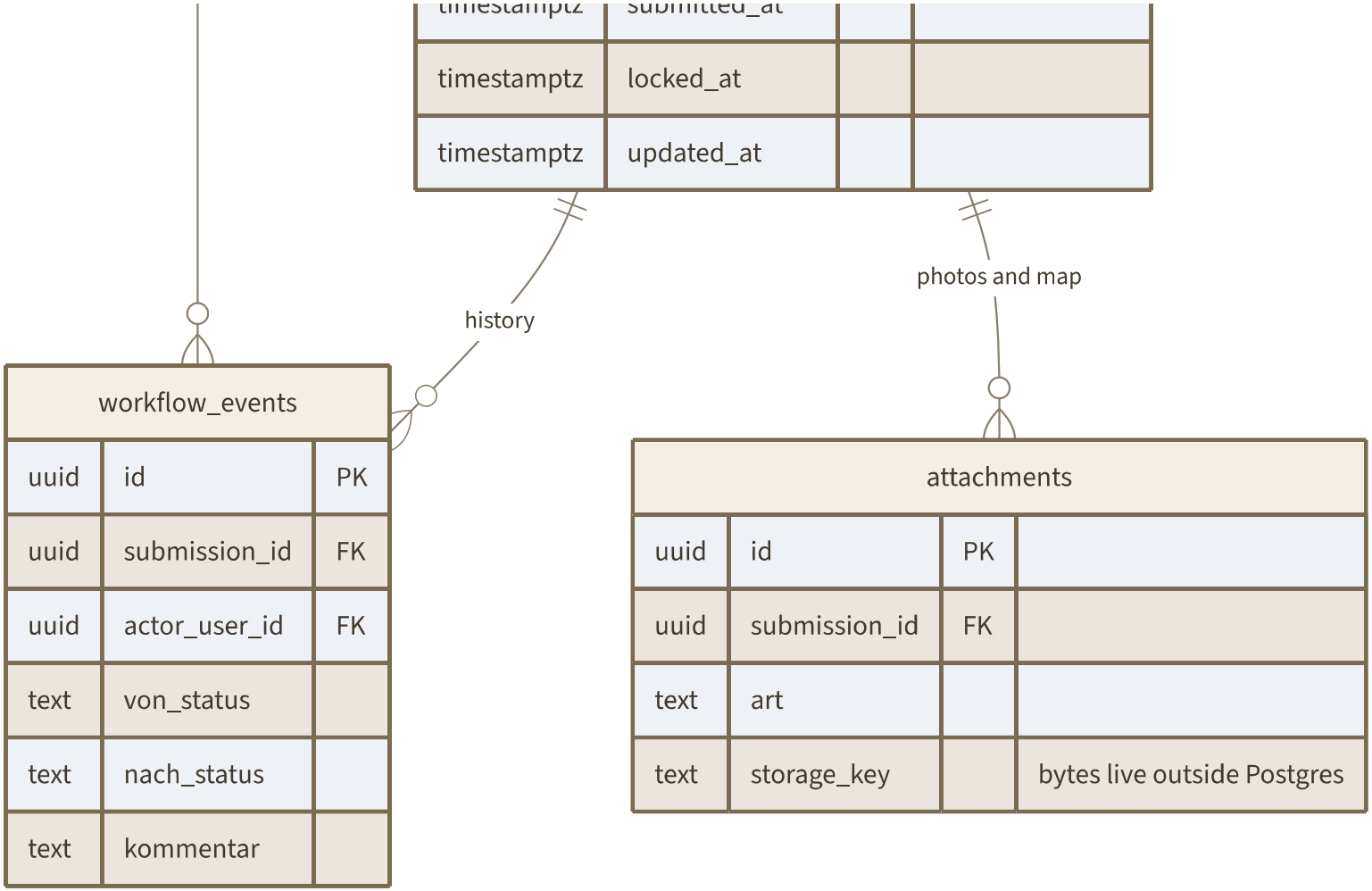
`Bearbeiter`
The person who carried out the survey.

`Anlage`
An attachment: a photo or the map excerpt.

2 THE TABLES




uuid
id
PK
text
email
text_array
rollen
bool
ist_aktiv
probestrecken
personen
files (owner_user_id)
surveyed at
Bearbeiter
made the move
submissions
uuid
id
PK
uuid
owner_user_id
FK
text
status
int
version
jsonb
antworten
all answers
text
form_version
uuid
probestrecke_id
FK
null while DRAFT
uuid
person_id
FK
null while DRAFT
date
datum
null while DRAFT
text
anlass
null while DRAFT



The `submissions` row is the centre. The envelope columns (stretch, person, date, Anlass) stay empty while the protocol is a draft and get filled in at submit.

3 UNDER THE HOOD

– Columns for what we search on, JSONB for everything else (ADR 0003)

The rule is simple: if the application filters, sorts or checks permissions on a value, it is a real column. If it is only ever shown back to a person, it lives inside

`antworten`.

Why: 338 columns would mean a migration every time FFS changes the form. One JSON document means the form can change without touching the schema, and the few values we query stay fast and typed.

Why JSONB and not plain JSON: Postgres can index and search inside JSONB. The review queue's "search by species" depends on that (see feature 2).

+ **Every request re-checks who you are**

+ **Ownership is part of the SQL query, and strangers get 404**

+ **Status and roles are text with check constraints, not Postgres ENUM types**

+ **Every rule is written twice: in the database and in Python**

FEATURE 1

Entwürfe speichern · saving drafts

`features 3a, 3b, 3c, 4a`

1 IN ONE MINUTE

Anna opens "Neues Protokoll". Nothing is saved yet, and nothing is created in the

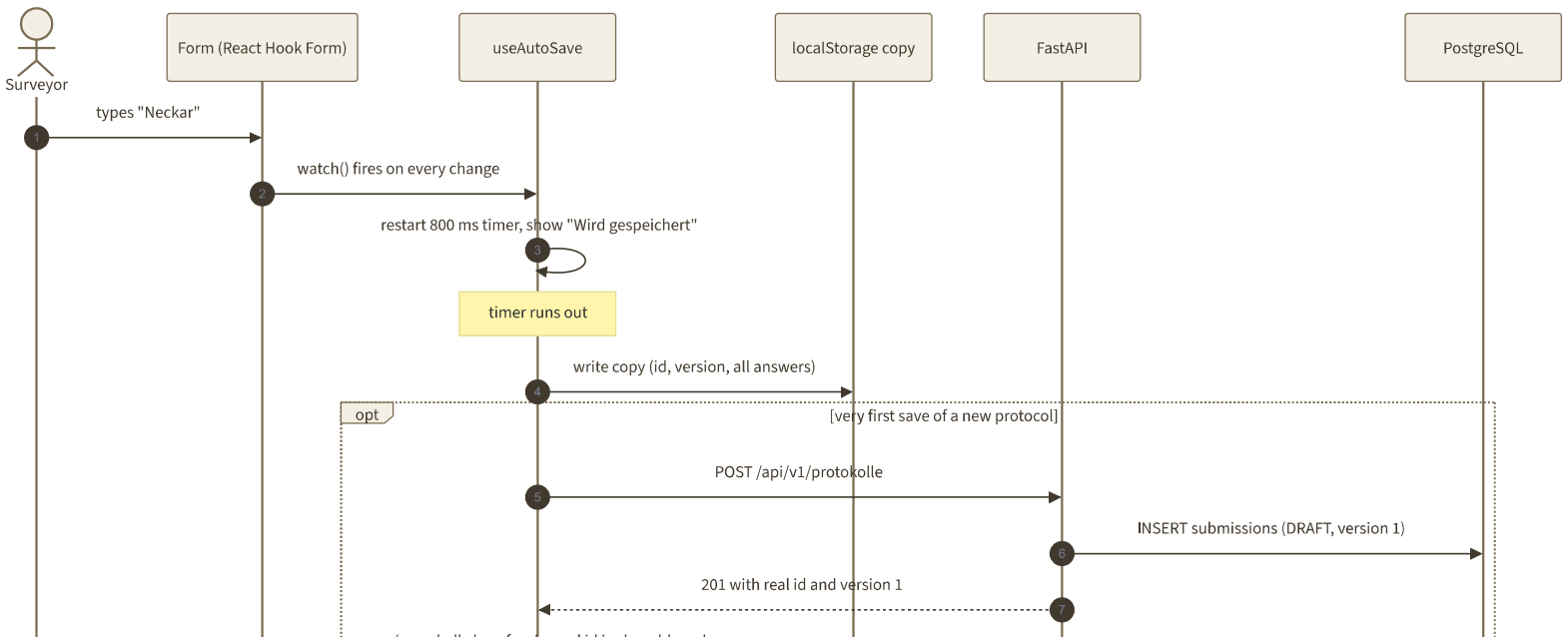
database. She types **"Neckar"** into the water name field.

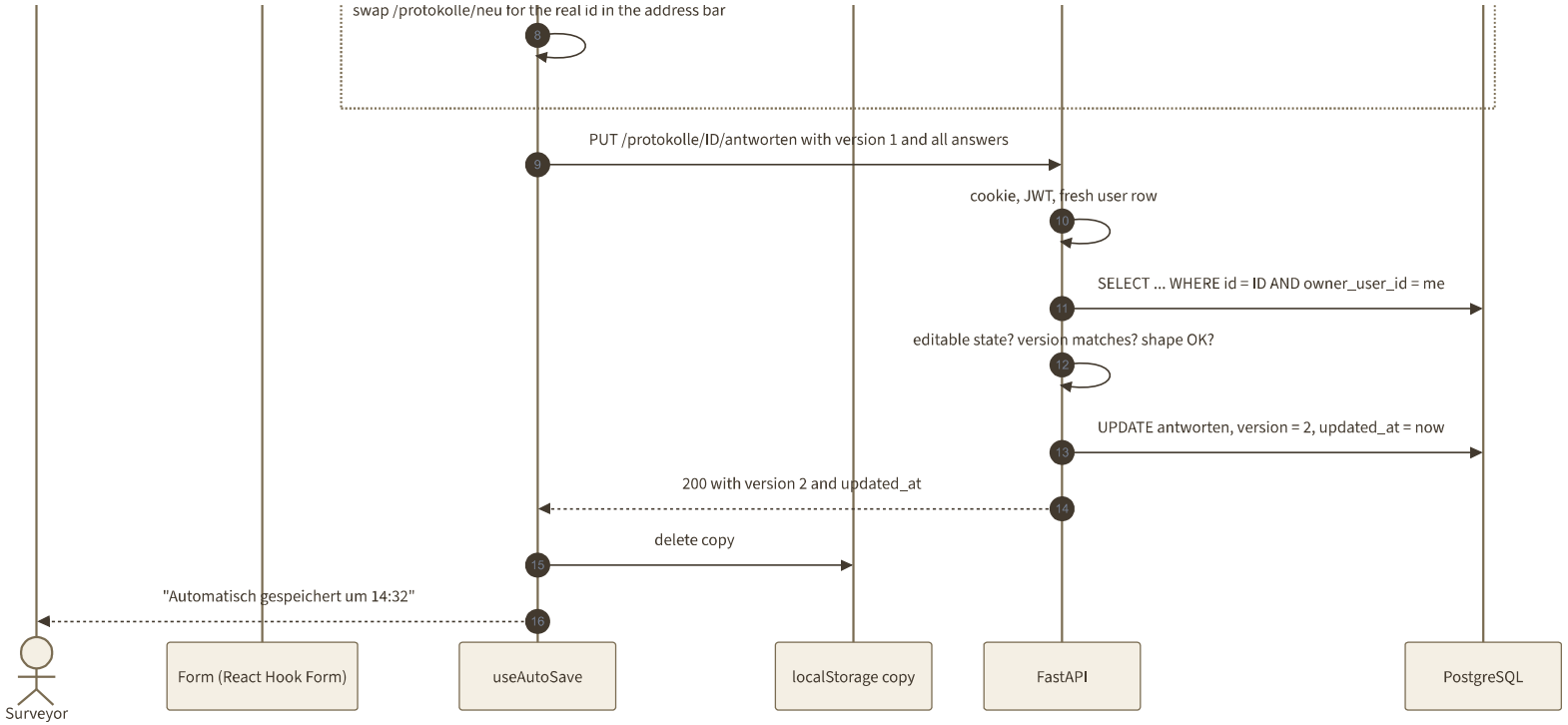
The app waits until she has stopped typing for **0.8 seconds**. Then it does two things: it puts a safety copy of everything in her browser, and it sends the whole form to the server. Because this is the first save, the server first creates her protocol, then stores the answers.

The server says "saved, you are now at version 2". The browser deletes the safety copy and shows "Automatisch gespeichert um 14:32".

Next day her wifi drops mid-save. The safety copy stays in the browser. When she reopens the protocol, the app offers to restore it. Later she has the protocol open in two tabs: the second tab's save is refused because its version number is out of date, so one tab cannot silently overwrite the other.

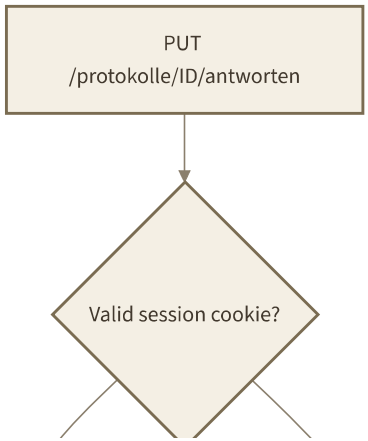
2 THE FLOW

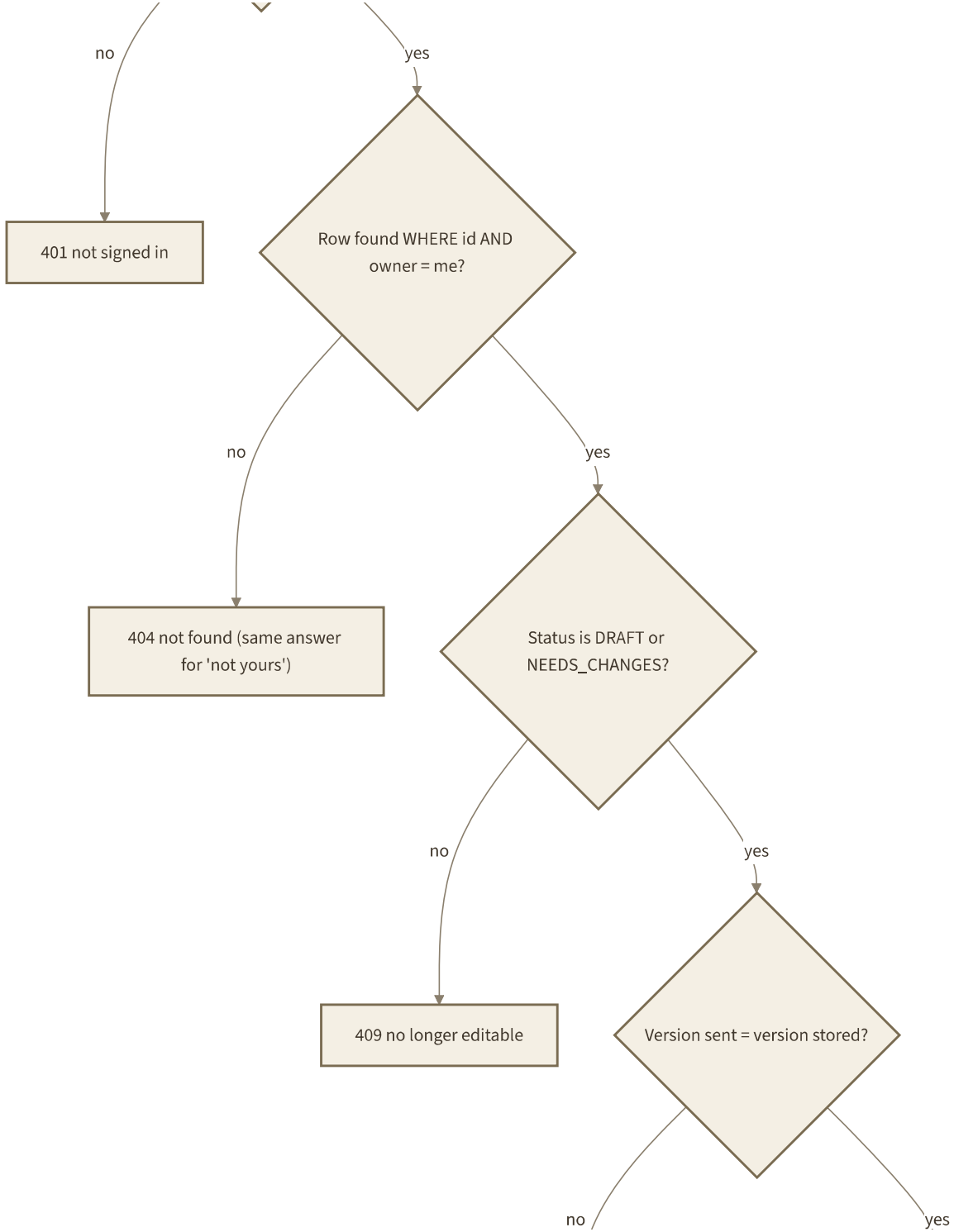


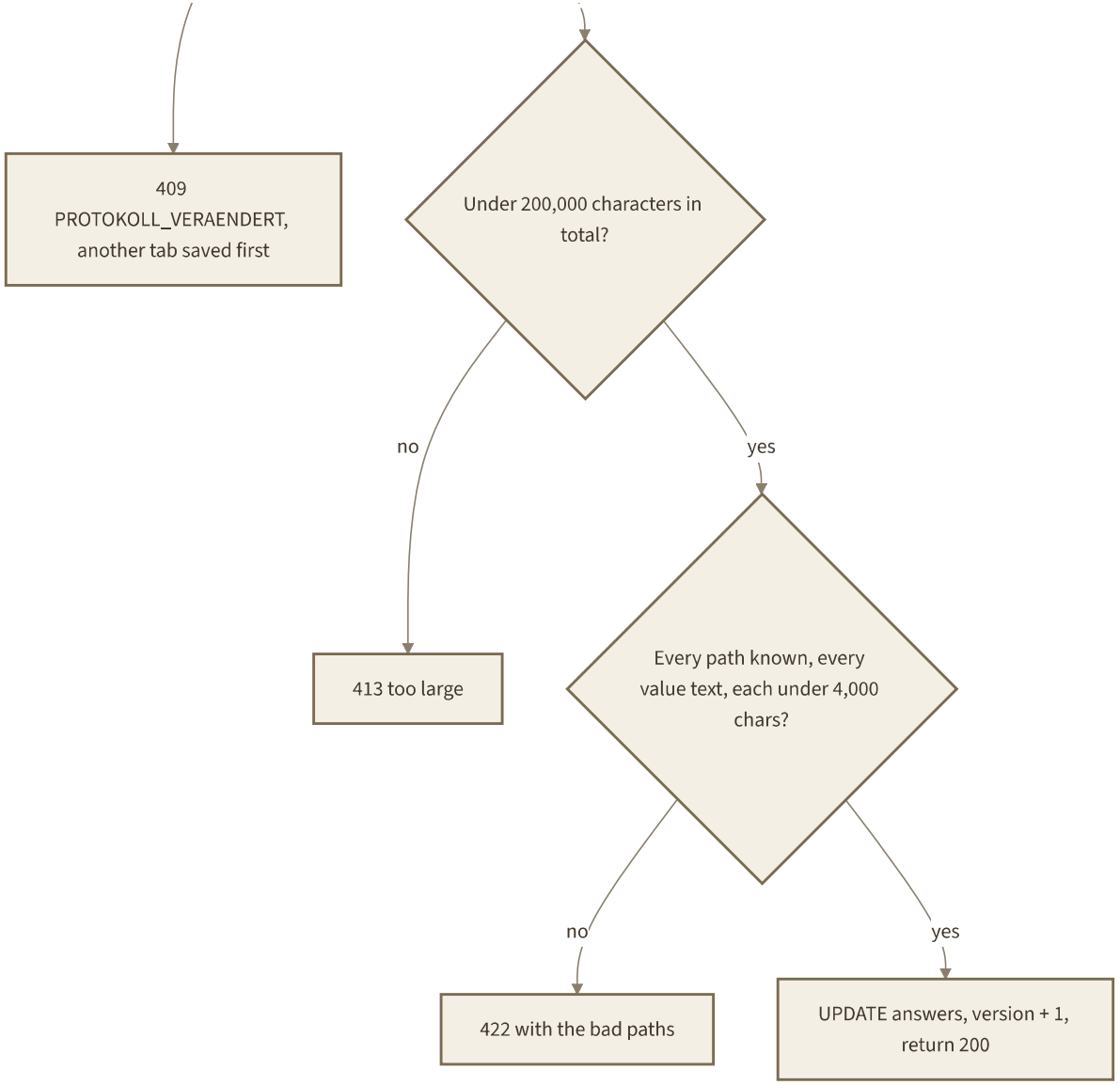


One save every pause in typing, never one per keystroke. Only one save is in flight at a time; anything typed meanwhile is picked up by the next one.

What the server checks on every save, in this order




no
yes
401 not signed in
Row found WHERE id AND owner = me?
no
yes
404 not found (same answer for 'not yours')
Status is DRAFT or NEEDS_CHANGES?
no
yes
409 no longer editable
Version sent = version stored?
no
yes



The order matters. The version is checked before the content, so someone with two tabs open is told about the second tab rather than about a field that is not really wrong.

3 UNDER THE HOOD

− Optimistic locking with a `version` column

Every row has an integer `version`, starting at 1 (a check constraint enforces `version >= 1`). The browser sends the version it is working from. If it does not match, nothing is written and the API returns 409.

On the frontend, 409 becomes its own state, `'conflict'`, not just "failed". After a conflict the hook stops saving entirely, because every retry would carry the same stale version.

Why "optimistic": we do not lock the row while someone types (that could take hours). We let everyone work and only refuse the save that arrives stale. Conflicts are rare, so this is cheap.

+ The whole document every time (PUT, not PATCH)

+ A draft save is deliberately forgiving

+ The local safety copy is written *before* the request

+ No empty protocols: the row is created by the first keystroke

+ Typing speed: 338 fields without lag

+ Timestamps: `updated_at` , `RETURNING` and `clock_timestamp()`

+ **"Meine Protokolle" reads display values straight out of the JSON**

+ **Deleting: drafts only, and files go too**

FEATURE 2

Rollen und Review · roles and review

`features 2b, 11a to 11f, 12a to 12d, 16`

1 IN ONE MINUTE

Anna (Submitter) presses **Absenden**. The server runs every form rule. Her percentages for "Umland" add up to 95, not 100, so the protocol is refused and the problem appears next to the right field. She fixes it and presses again.

This time it passes. The protocol becomes **SUBMITTED**. At that moment the server also files it properly: it finds or creates the water body "Neckar", the exact stretch she fished, and her as the Bearbeiter, and links the protocol to them.

Ben (Reviewer) sees it in the review queue. He clicks "In Prüfung nehmen" (IN_REVIEW), notices a wrong coordinate and chooses "Änderung anfordern" with the reason "Koordinaten prüfen". The protocol goes back to Anna as **NEEDS_CHANGES**, she can edit it again, and she resubmits. Ben accepts: the protocol is **LOCKED** for good. Every one of these moves is written into the history with who, when and why.

Ben could never accept his own protocol, and a Data Steward can read it but

cannot decide.

2 THE FLOW

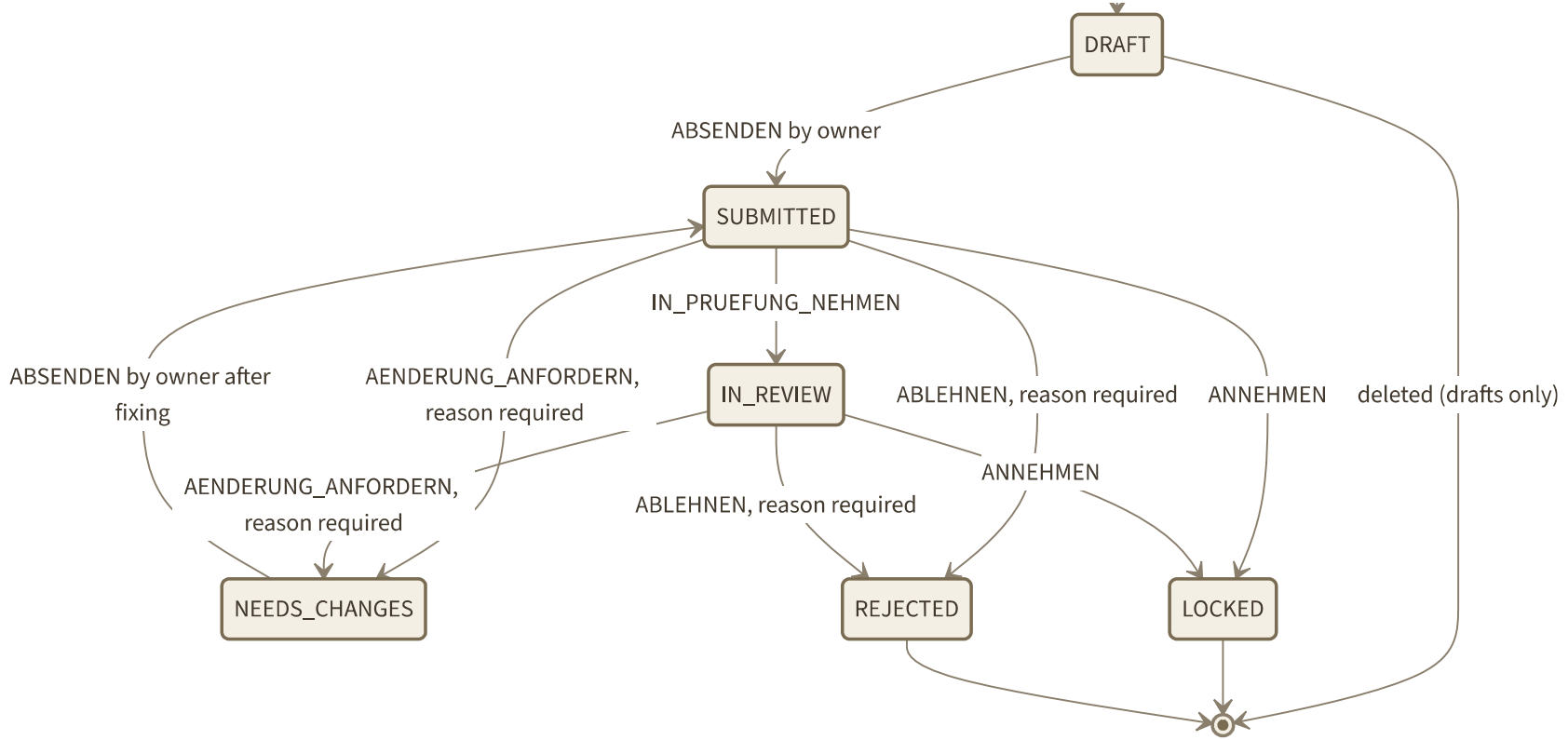
Who can do what

Role	Sees	Can do today
`SUBMITTER`	Only own protocols, any state	Create, save, delete drafts, submit, resubmit after a change request, download PDF
`REVIEWER`	Own protocols, plus everyone's that are not drafts	Review queue, take into review, accept, request changes, reject
`DATA_STEWARD`	Same as Reviewer	Read and quality-check, but **not** decide (chosen 2026-09-14)
`SUPER_ADMIN`	Same as Reviewer	Decide like a Reviewer, plus manage accounts and roles
`REGIERUNGSPRAESIDIUM`	Planned: own region, read-only	Not built yet (feature 13). Today it sees only its own protocols, on purpose
`INTEGRATION`	Planned: machine account	Not built yet (feature 19, transfer to FiaKa)

Nobody sees someone else's draft. Nobody decides on a protocol they filed themselves.

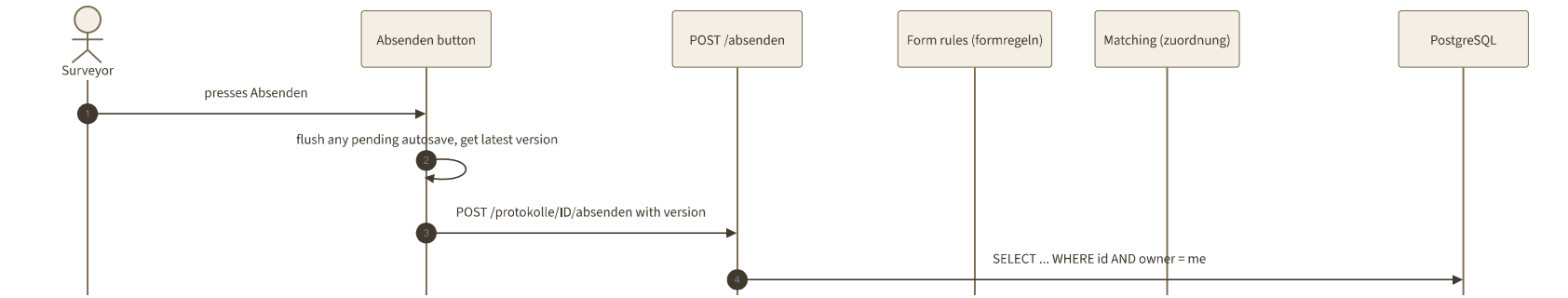
The life of a protocol

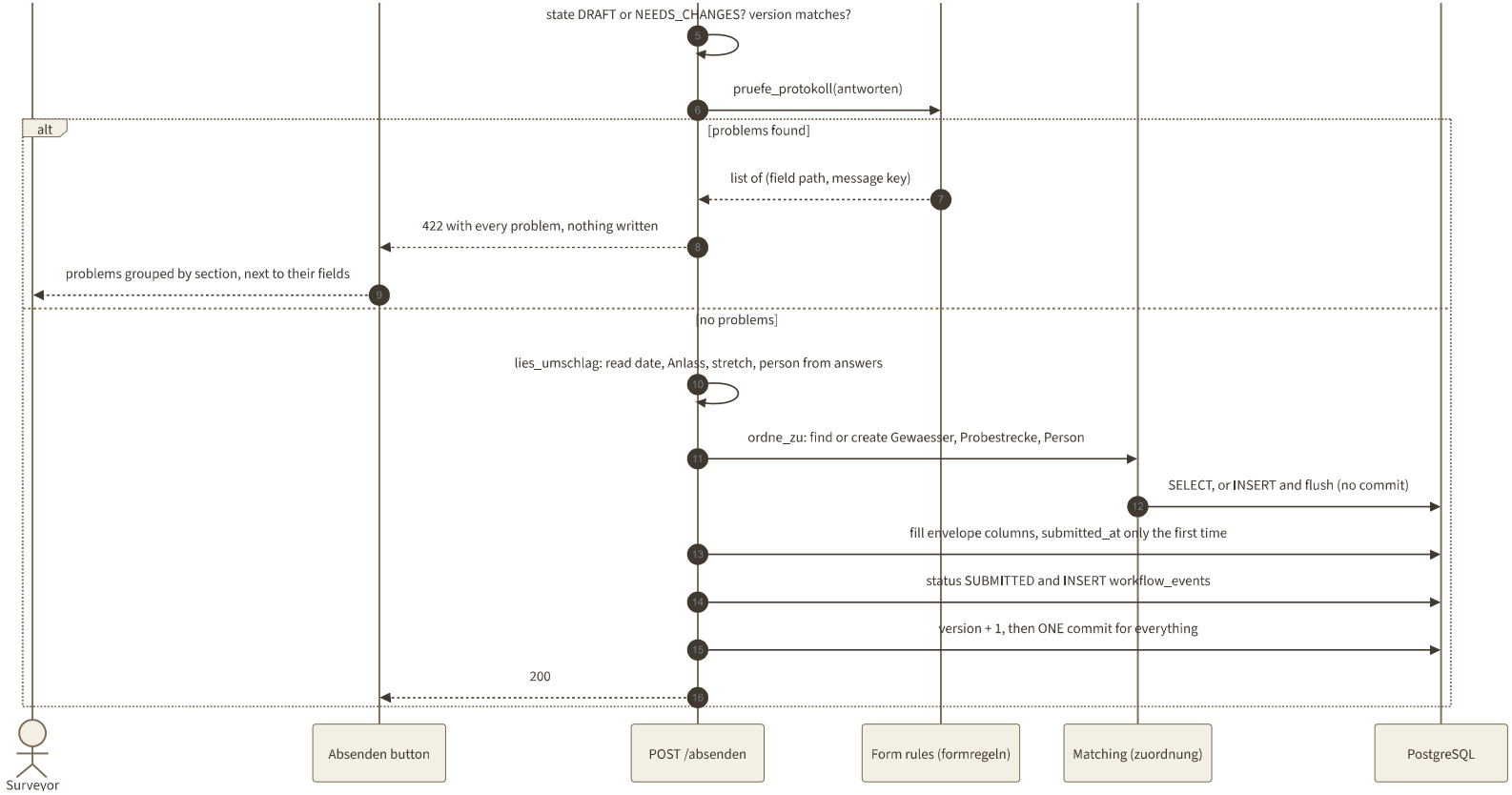




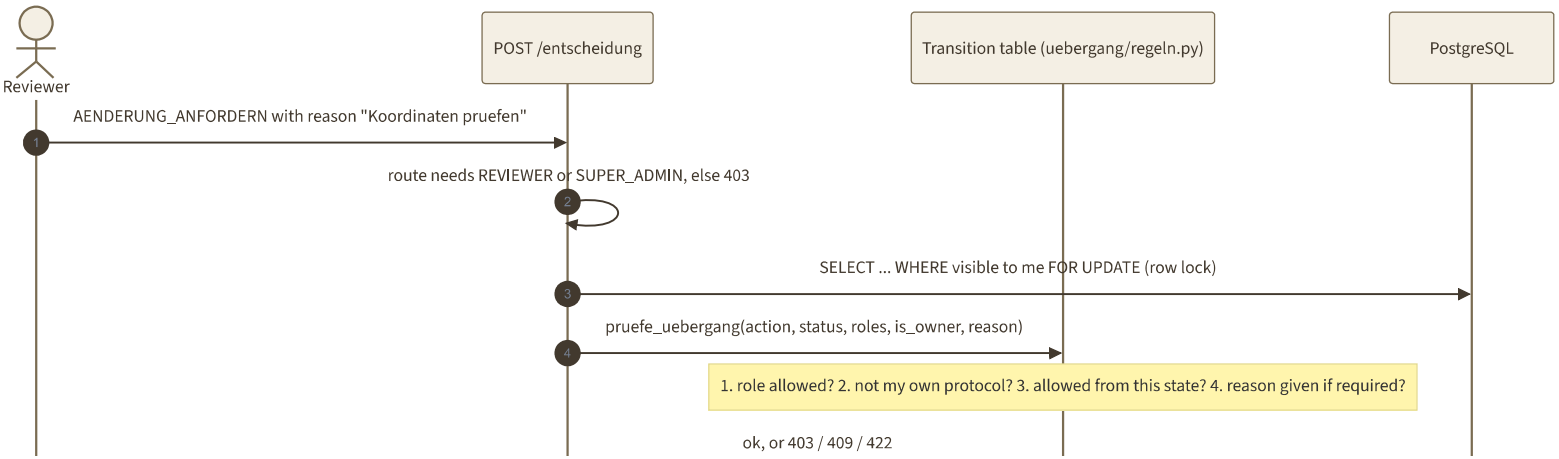
LOCKED and REJECTED are final. `ACCEPTED` exists in the enum but nothing writes it yet: accepting goes straight to LOCKED until the FiaKa transfer (feature 19) needs a step in between.

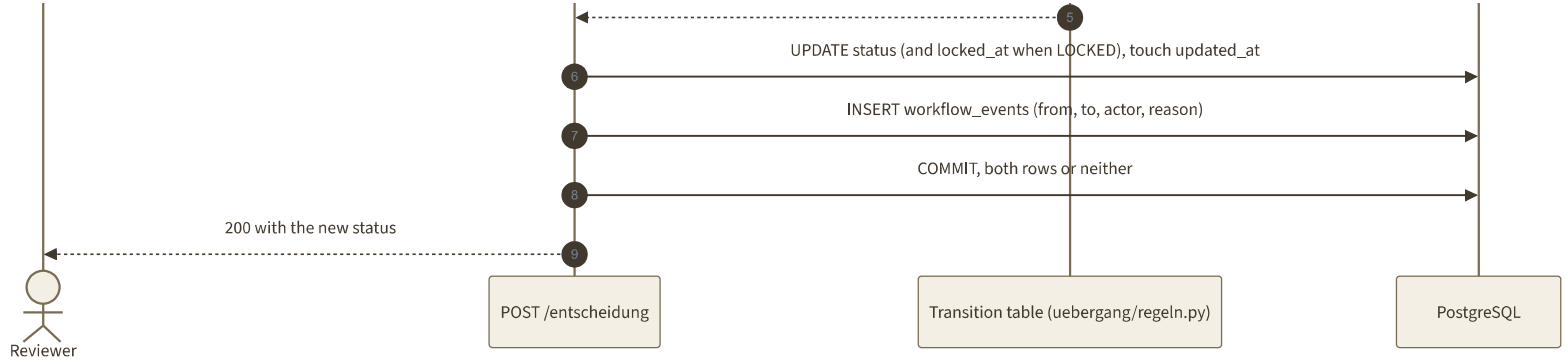
Submitting (Absenden)





A reviewer decides





3 UNDER THE HOOD

− The state machine is a table of data, not a pile of if-statements

`UEBERGAENGE` in `protokolle/uebergang/regeln.py` lists each action with: which states it may start from, which state it produces, which roles may do it, and whether a reason is required. One function, `pruefe_uebergang()`, reads that table. Routes take their role list from the same table, so there is exactly one answer to "who may accept".

Actions are named after what the person does (`ANNEHMEN`), not the target state. An API that accepted "set status to X" would one day be asked for `LOCKED` by the wrong person.

No database, no HTTP in this file, so the whole state machine is unit-tested as a table.

+ Status change and history row are one transaction

+ A row lock (`SELECT ... FOR UPDATE`) stops two reviewers colliding

+ **Nullable columns that are not optional:** `umschlag_bei_abgabe`

+ **Matching at submit: read or create, never update**

+ **Partial unique indexes let the database win the race**

+ **Rules exist twice: TypeScript for instant feedback, Python as the gate**

+ **Visibility rules live in one function,** `_sichtbar()`

+ **The review queue: columns, a GIN index, and a window function**

FEATURE 3

PDF-Import · reading the old form in

`features 23a, 23b, 23c (23d, the pictures, not built yet)`

1 IN ONE MINUTE

Carl filled in the old Acrobat PDF on his laptop, as he has done for years. Instead of typing it all again, he uploads the file on "Meine Protokolle".

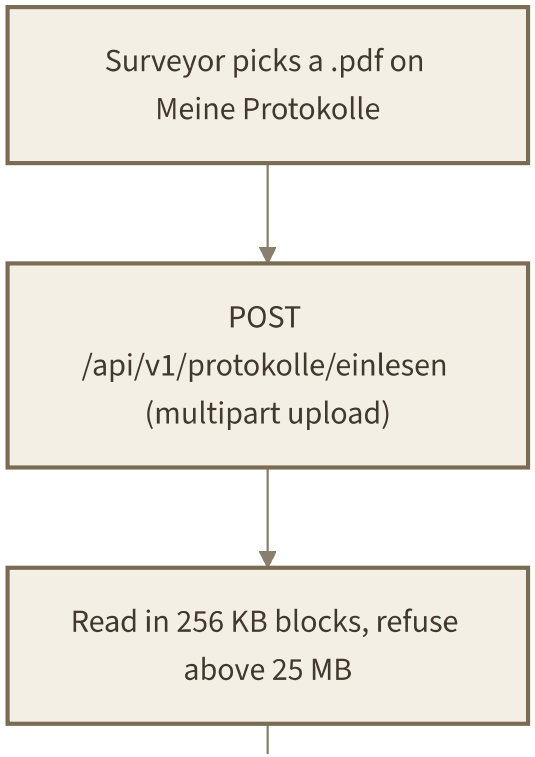
The server checks that it really is this form, reads every field, and converts

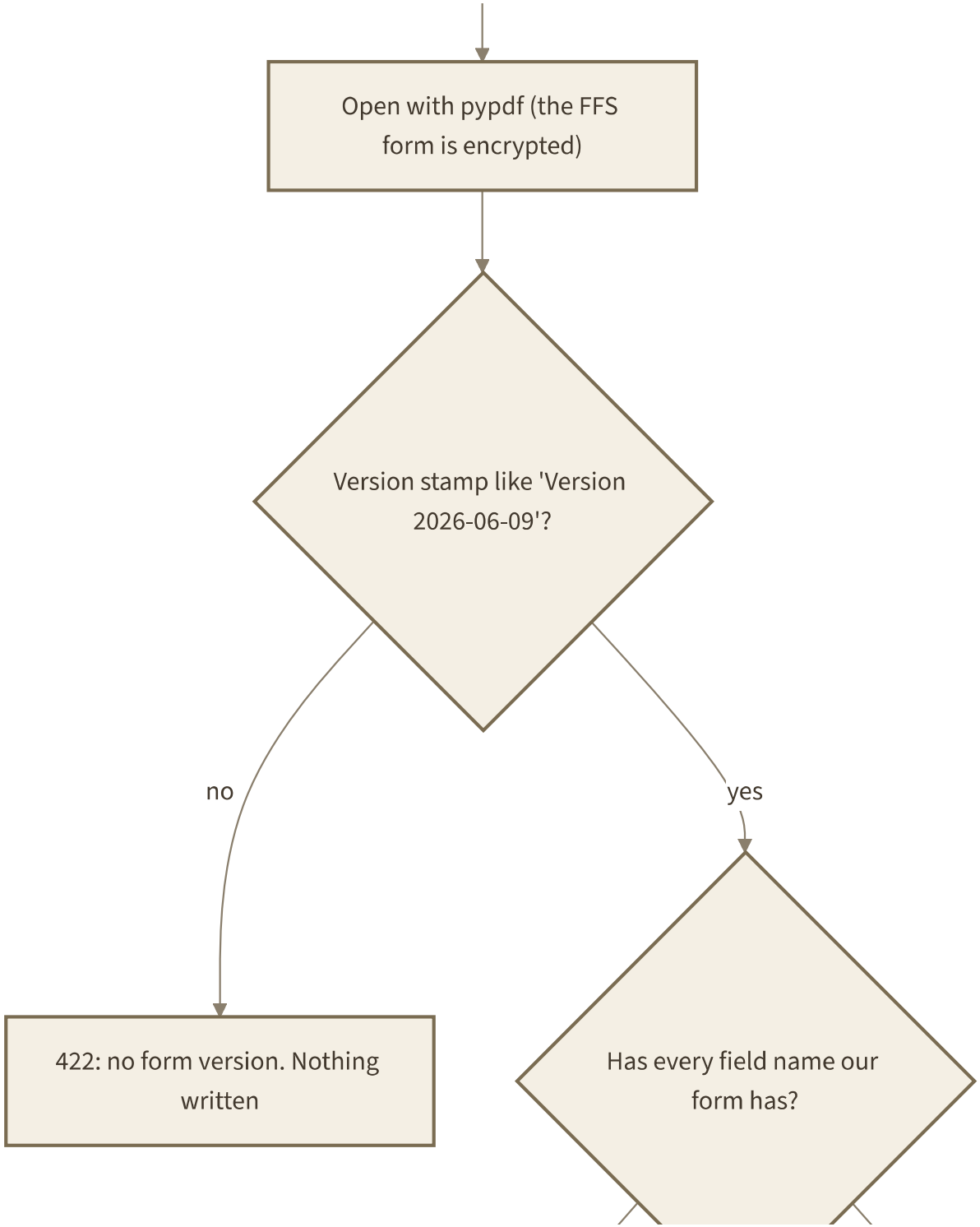
German writing into ours: **"1.234" fish becomes 1234**, not 1.2, and **"04.05.2026" becomes 2026-05-04**. It creates a new **draft** that belongs to Carl, with all answers filled in.

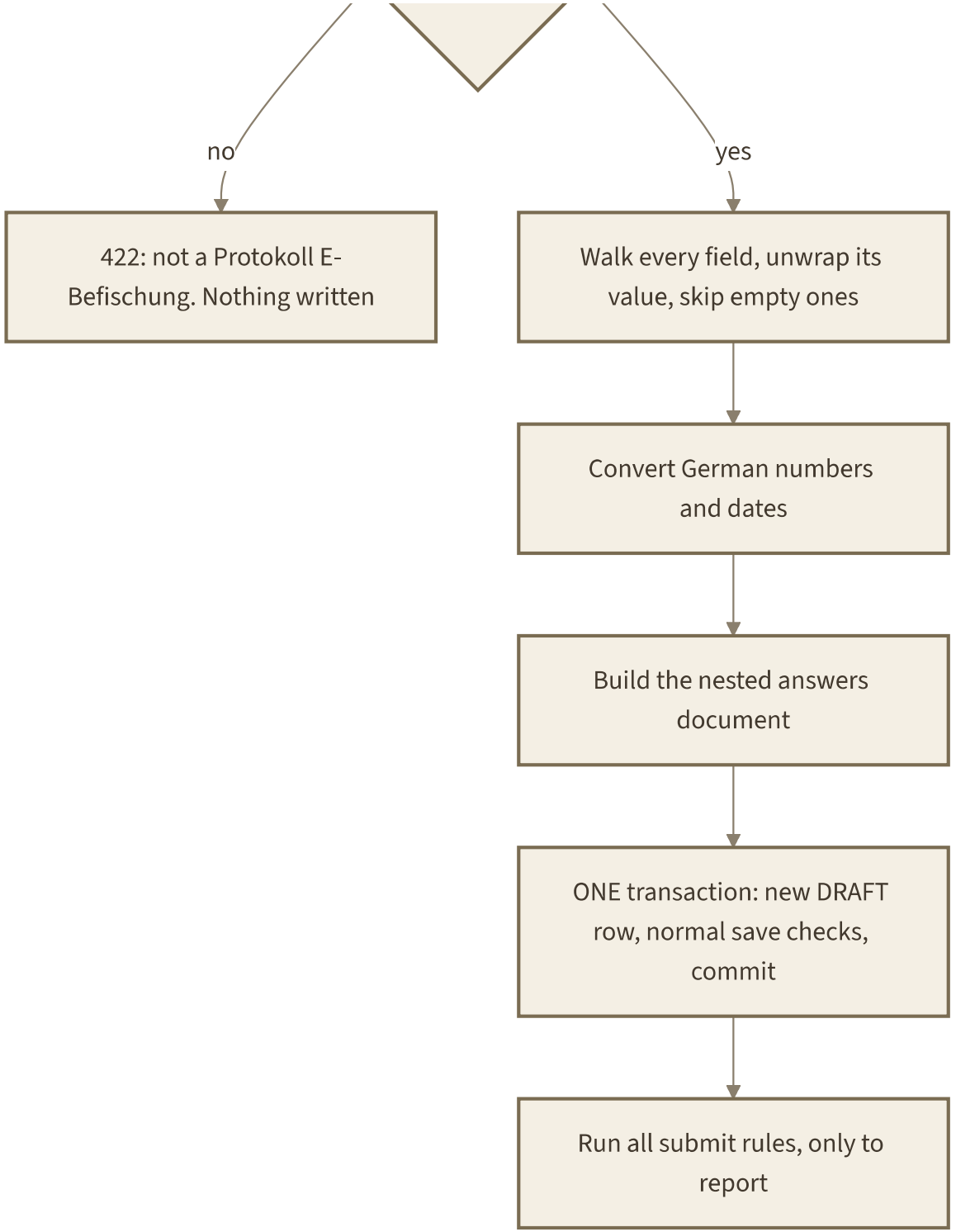
Then it runs the same rules as Absenden, but only to *report*. Carl lands on section 1 with a banner ("this came from a PDF") and a list of what is still wrong. Nothing is submitted for him. The old form has known validation bugs, so an import is never trusted.

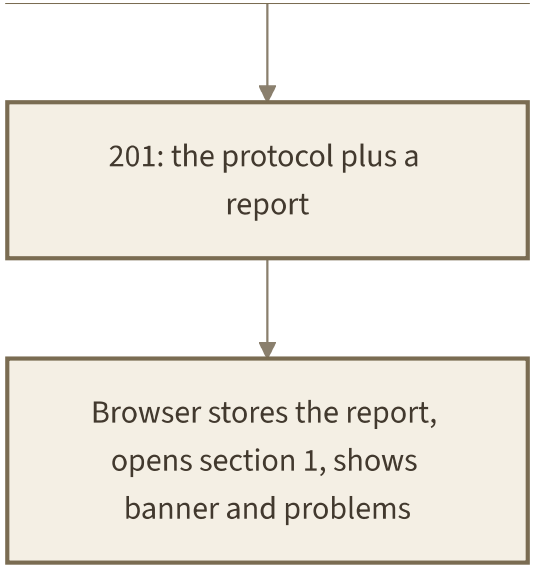
If he uploads the crayfish form by mistake, the server says so and writes nothing at all.

2 THE FLOW




Open with pypdf (the FFS form is encrypted)
Version stamp like 'Version 2026-06-09'?
no
yes
422: no form version. Nothing written
Has every field name our form has?


no
yes
422: not a Protokoll E-Befischung. Nothing written
Walk every field, unwrap its value, skip empty ones
Convert German numbers and dates
Build the nested answers document
ONE transaction: new DRAFT row, normal save checks, commit
Run all submit rules, only to report



The report has five parts: the form version the file declared, problems from the rules, values that could not be read, how many pictures the file held, and (logged only, not shown) fields we have no home for.

3 UNDER THE HOOD

– Numbers are the dangerous part (`einlesen/werte.py`)

373 of the form's 383 number fields use a dot for thousands and a comma for decimals. Read naively, "1.234" fish become 1.2 fish and 1,232 fish disappear without any error.

- A dot counts as a thousands separator only if it is followed by **exactly three digits every time**. So "1.234" is 1234, but "12.5" stays "12.5" and the catch rule later rejects it as not a whole number.
- Every conversion is safe to run on a value that is already in our format, because we cannot tell which of Acrobat's two formats the file stored.

- A value that cannot be read is **kept exactly as written and flagged**, never replaced by something plausible. 31.02.2026 is kept and flagged, not rolled into March.

+ **Check what the file is before reading any answers**

+ **Same write path as a normal save, in one transaction**

+ **Unwrapping PDF field values**

+ **The report survives a reload, in the browser**

+ **Pictures: counted now, extracted later (23d)**

+ **Safety limits**

FEATURE 4

PDF-Export · the protocol as a PDF

`feature 23e`

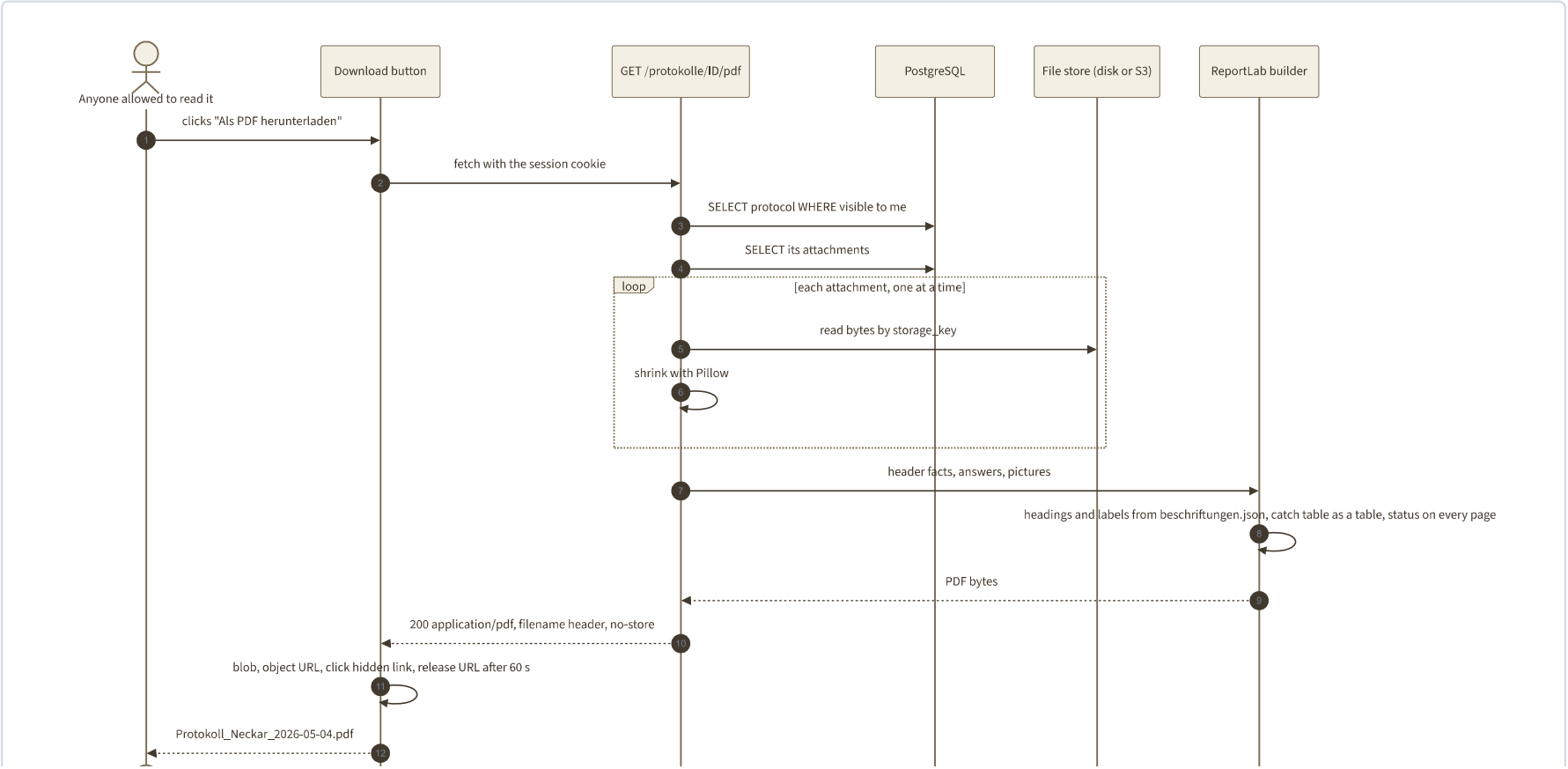
1 IN ONE MINUTE

Anna wants a copy to keep and to send to the landowner, who has no account. She clicks **"Als PDF herunterladen"**.

The server collects the protocol's answers and its photos, lays them out as a clean A4 document with the same headings and labels as the screens, and sends it back. The file arrives as **Protokoll_Neckar_2026-05-04.pdf**.

It works for drafts too. That is fine, because the status (for example "Entwurf") is printed on every page, so a draft can never be mistaken for a filed protocol. Ben can download Anna's submitted protocol as well, since he is allowed to read it.

2 THE FLOW





3 UNDER THE HOOD

− Our own document, not a filled-in copy of the official form

Decided on 2026-09-23. The backend never opens the legacy PDF to export, so the blank FFS form is not needed at run time. Our own page also has room for things the paper form has no box for: the Bearbeiter's town, more than four photos, and the status.

+ One source for labels: generated from the frontend

+ No rules run, and empty fields are left out

+ Same permission as reading, with the same 404

+ Memory and missing files

+ ReportLab, built whole, never cached

REFERENCE

Cheat sheet

Endpoints

Endpoint	Feature	What it does
`POST /protokolle`	Drafts	Create an empty draft (called by the first save)
`PUT /protokolle/{id}/antworten`	Drafts	Replace all answers, with version check
`GET /protokolle`	Drafts	My protocols, summary rows only
`DELETE /protokolle/{id}`	Drafts	Delete a draft and its files
`GET /protokolle/{id}`	All	One protocol in full, if visible to me
`POST /protokolle/{id}/absenden`	Review	Submit: rules, matching, envelope, history
`POST /protokolle/{id}/pruefung`	Review	Take into review
`POST /protokolle/{id}/entscheidung`	Review	Accept, request changes, or reject
`GET /protokolle/{id}/verlauf`	Review	History, newest first
`GET /pruefliste`	Review	Review queue with filters, species search, paging
`POST /protokolle/einlesen`	Import	Upload a legacy PDF, get a draft and a report
`GET /protokolle/{id}/pdf`	Export	Download the protocol as a PDF

All under `/api/v1`. Full interactive docs at `http://localhost:8000/api/v1/docs` while the stack runs.

Where to start reading the code

Feature	Backend	Frontend
Drafts	`app/protokolle/dienst.py`, `app/protokolle/regeln.py`, `app/models/protokoll.py`	`protokoll/entwurf/useAutoSave.ts`, `sicherung.ts`, `neu.ts`
Roles and review	`app/api/abhaengigkeiten.py`, `app/protokolle/absenden.py`, `uebergang/`, `zuordnung/`, `formregeln/`, `pruefliste/`	`protokoll/absenden/`, `protokoll/pruefung/`, `protokoll/pruefliste/`, `auth/startseite.ts`
Import	`app/protokolle/einlesen/` (start at `protokoll.py` and `dienst.py`), `app/formular/pdf.py`	`protokoll/einlesen/useEinlesen.ts`
Export	`app/protokolle/ausgabe/` (`dienst.py`, `dokument.py`)	`protokoll/ausgabe/useHerunterladen.ts`, `scripts/beschriftungen.ts`

The ideas that repeat everywhere

- **Permissions in the `WHERE` clause**, and 404 for anything you may not see.
- **One transaction per user action**: submit, decide and import either fully happen or leave nothing behind.
- **The database guarantees, Python explains**: check constraints and unique indexes hold the line, Python gives the readable message.
- **Plain functions for rules** (`regeln.py` files): no database, no HTTP, easy to unit test.
- **One source of truth** for fields (`felder.json`), labels

- **One source of truth** for fields (`felder.json`), labels (`beschriftungen.json`) and transitions (`UEBERGAENGE`).

Written from the code on main as of 2026-10-04 (after feature 17e). The PlantUML diagram in `blueprint/history/flow_diagrams/DATA_FLOW_DIAGRAM.puml` traces the autosave path in even more detail.